

# Speech and language technology (DA 3452)

...

Lecture 5 – Aug 12  
TF-IDF

# Recap : Why not pure token system for whole document tasks ?

A token sequence is usable—but awkward for many whole-document tasks.

## 1. Different lengths

$D_1 \rightarrow [12, 4, 91, 17]$   
 $D_2 \rightarrow [7, 8, 45, 91, 6, 2, \dots]$   
 $D_3 \rightarrow [\dots 150 \text{ tokens } \dots]$

Documents produce sequences of different lengths.

## 2. IDs are labels

417 = "new"  
68 = "e"

417 is not larger, more important, or semantically "farther" than 68.

## 3. Whole-document questions

What is the topic of the document?  
Which document matches this query?  
Which two documents are similar?

These questions treat the whole document as one object.

Useful target form:

$D_1 \rightarrow [x_1, x_2, \dots, x_{|V|}]$     $D_2 \rightarrow [x_1, x_2, \dots, x_{|V|}]$     $D_3 \rightarrow [x_1, x_2, \dots, x_{|V|}]$

same feature space, same coordinate meanings, regardless of document length

A BPE sequence is valid, but document lengths vary and token IDs are categorical labels.  
If we want to compare, score, or classify documents, a fixed feature space is often useful.

# Recap : Motivation for document vectors – Document Search System

## DOCUMENT-LEVEL TASK

A search system begins with:

A collection of documents

A user query

A way to compare query and documents

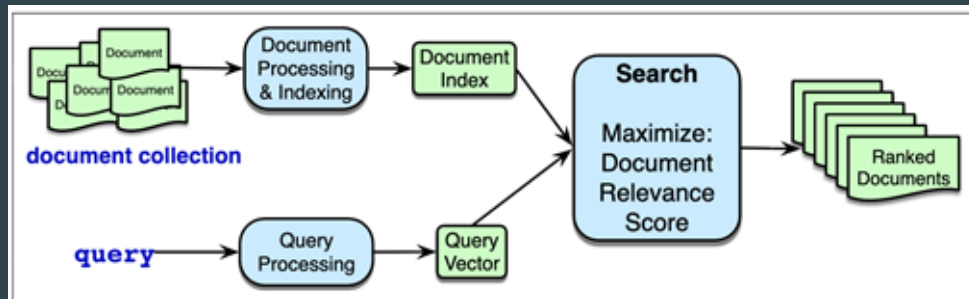
That comparison becomes easier if:

Each document has a common representation

The query has a compatible representation

One classical solution:

Bag-of-words (BoW) + TF-IDF vector



**Figure 11.1** The architecture of an ad hoc IR system. Document ranking is based on computing a score for each candidate document given the query, expressing how **relevant** it is likely to be to meet the users information need. There are two classes of IR systems, based on the two classes of vectors that are used to represent queries and documents: sparse vectors and dense vectors. These two kinds of retrieval differ in the details of the indexing and scoring mechanisms.

We now study one classical way to convert a whole document into a fixed-length vector – BoW + TF-IDF

# Recap : Bag of Words (BoW)

## Bag-of-Words (BoW)

Represent a document by which selected features occur and how often.

“Words” are convenient here; we could also count BPE tokens or characters.

Example Sentence : “the batsman hit the ball to the boundary”

Note that only the **counts matter** but **order is not accounted for**

### 1. Ordered tokens

the batsman hit the ball to the boundary

### 3. Counts

|          |   |
|----------|---|
| the      | 3 |
| batsman  | 1 |
| hit      | 1 |
| ball     | 1 |
| to       | 1 |
| boundary | 1 |

### 4 . Fixed feature order → vector

[the, batsman, hit, ball, to, boundary]

$D_1 \rightarrow [3, 1, 1, 1, 1, 1]$

### 2. Unordered Bag

ball the boundary the  
batsman the to hit

# Recap : Turning document similarity into a geometric problem

DOC 1 CRICKET

“the batsman hit the ball  
to the boundary”

DOC 2 CRICKET

“the bowler bowled the  
ball at the batsman”

DOC 3 MARKET

“the vendor sold sweet  
mangoes in the market”

DOC 4 MARKET

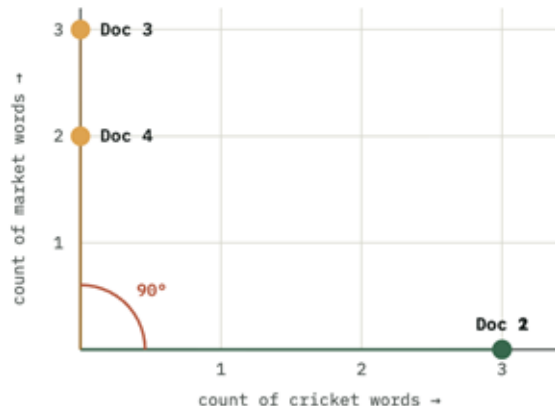
“the price of mangoes  
rose in the market”

For intuition, compress the real vocabulary-sized space to two grouped axes – **cricket & market**

$D_1$  and  $D_2$  point in similar  
directions.

$D_3$  and  $D_4$  point in similar  
directions.

Cricket and market  
documents point in very  
different directions.



## Cosine similarity:

“How similar are the directions of two  
document vectors?”

same direction  $\rightarrow$  cosine  $\approx 1$

right angle  $\rightarrow$  cosine  $\approx 0$

For intuition only — we will return to vector similarity  
later.

$$\cos(d_1, d_2) = (d_1^T d_2) / (\|d_1\| \|d_2\|)$$

Coordinates here are counts of cricket words (batsman, bowler, ball, boundary) and market words (vendor, mangoes, market) – a 2-D shadow of the real, vocabulary-sized space.

# Module 1

## Representing Language Computationally

Document Vectors

# Document vector

In Language processing, “Document” does not necessarily mean an entire file. It is the unit for the task.

document-term matrix

|                | the | ball | batsman | in | mangoes | market | boundary | bowler | hit | price | vendor |
|----------------|-----|------|---------|----|---------|--------|----------|--------|-----|-------|--------|
| D <sub>1</sub> | 3   | 1    | 1       | .  | .       | .      | 1        | .      | 1   | .     | .      |
| D <sub>2</sub> | 3   | 1    | 1       | .  | .       | .      | .        | 1      | .   | .     | .      |
| D <sub>3</sub> | 2   | .    | .       | 1  | 1       | 1      | .        | .      | .   | .     | 1      |
| D <sub>4</sub> | 2   | .    | .       | 1  | 1       | 1      | .        | .      | .   | 1     | .      |

## Document

The unit of text we choose to represent as one object

### Task-dependent document units

review

email

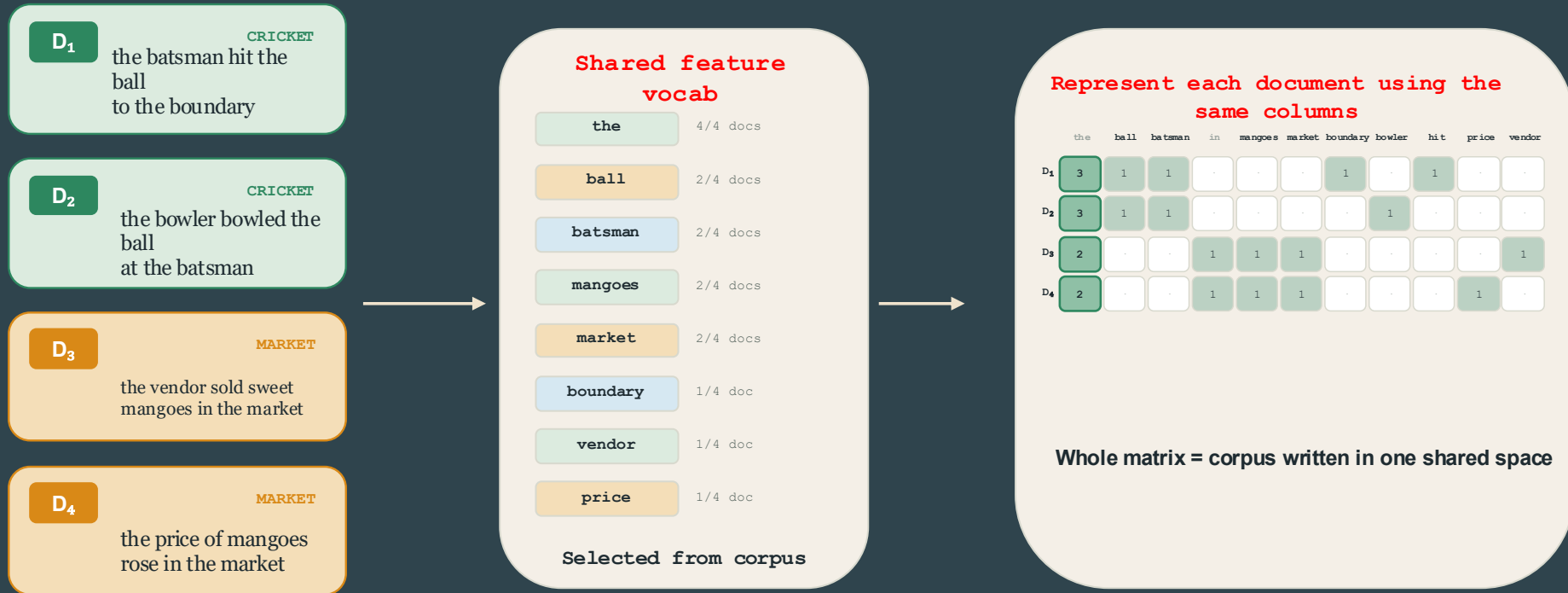
article

transcript

If each row is a “document”, what is the whole collection of documents?

# A corpus is the collection that gives us the shared feature space

The **corpus** defines a shared feature **vocabulary**, so every **document** can be represented in the same coordinate system.



Are all columns equally useful? That is, are all features equally useful?

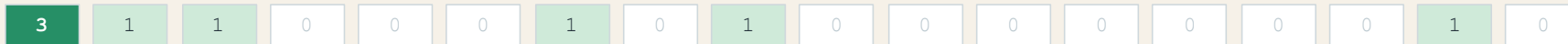


# Most coordinates in a document vector are zero

A large feature vocabulary gives every document many possible coordinates; a single short document uses only a few -> sparsity

$D_1$  = the batsman hit the ball to the boundary

Dense vector over the full 18-word corpus vocabulary



Non-zero entries are highlighted; zeros are faded.

## Sparse view

the → 3  
ball → 1  
batsman → 1  
boundary → 1  
hit → 1  
to → 1

## Storage choice

### Dense storage

Store every coordinate, including zeros.

### Sparse storage

Store only the non-zero feature/value pairs.

## Corpus-scale intuition

Small classroom vocabulary → some zeros

Realistic vocabulary → tens of thousands of features

One short document uses only a tiny fraction

Sparse storage changes efficiency — not the mathematical representation.

Are all columns equally useful? That is, are all features equally useful?

# The tyranny of “the”

Raw counts treat every word occurrence equally.

But some words occur everywhere while helping us distinguish topics very poorly.

In this corpus, “the” contributes little topical discrimination.



Doc 1 · cricket

vs

Doc 2 · cricket

cosine = 0.79

Doc 1 · cricket

vs

Doc 3 · market

cosine = 0.51

Why is this a problem?

Example: Every document contains “the”



Every vector gets a large “the” component



Unrelated documents can look more similar than reqd

Same topic cosine = 0.79

Different topic cosine = 0.51

Not much difference between the two due to uninformative “the”

Perhaps different words should receive different weights?

# What should make a word important for a document?

A useful weight should combine what happens inside one document with what happens across the corpus.

## Frequent HERE

$D_1$ : the batsman hit the ball to the boundary

the

3

ball

1

batsman

1

boundary

1

More occurrences here →  
potentially more weight

Useful document weight

≈

frequent HERE

×

rare ELSEWHERE

## Rare ELSEWHERE

the

appears in 4 / 4 documents

mangoes

appears in 2 / 4 documents

boundary

appears in 1 / 4 document

Fewer documents → more  
discriminative

**This is the intuition behind TF-IDF.**

First approximation: Common words within multiple documents should receive less weight.

“the” occurs in 4/4 documents; “boundary” occurs in 1/4 document. How can we turn these into numerical weights?

# Term Frequency (TF): What happens HERE?

Intuition — A useful weight depends partly on what happens inside the current document.

## Frequent HERE → Term Frequency (TF)

### Local count in one document

$c(t, d)$  = number of times term  $t$  occurs in document  $d$

There are several TF conventions. For this course, we use log-weighted TF.

$$\begin{aligned} \text{tf}(t, d) &= 0 && \text{if } c(t, d) = 0 \\ \text{tf}(t, d) &= 1 + \log_{10}(c(t, d)) && \text{if } c(t, d) > 0 \end{aligned}$$

For  $D_1$ : “the batsman hit the ball to the boundary”

|          |   |                                  |
|----------|---|----------------------------------|
| the      | 3 | $1 + \log_{10}(3) \approx 1.477$ |
| ball     | 1 | $1 + \log_{10}(1) = 1$           |
| boundary | 1 | $1 + \log_{10}(1) = 1$           |

Repeated occurrence increases TF, but not linearly.

TF measures strength within this document.

TF is local to one document.

# Document Frequency (DF): What happens across the corpus?

**Intuition** — A useful weight also depends on how widely a term is spread across the corpus.

## Across the CORPUS → Document Frequency (DF)

How widely is the term spread across the corpus?

$df_t$  = number of documents containing term  $t$

Each document contributes at most 1 to DF, even if the term appears many times inside it.

**For the 4-document classroom corpus**

|          |        |                             |
|----------|--------|-----------------------------|
| the      | df = 4 | appears in 4 of 4 documents |
| ball     | df = 2 | appears in 2 of 4 documents |
| boundary | df = 1 | appears in 1 of 4 documents |

Higher DF means the term is more common across documents.

DF  $\uparrow$   $\Rightarrow$  more common across the corpus

DF measures **corpus-wide prevalence**.

Common across many documents  $\rightarrow$  high DF.

# Inverse Document Frequency (IDF)

Intuition — DF tells us how common a term is across documents. For weighting, we want the opposite effect.

Common across MANY documents → lower weight

- 1 DF is large for common terms.
- 2 Common terms should get less weight.
- 3 So we invert DF.

$$\text{idf}_t = \log_{10} ( N / \text{df}_t )$$

Here,  $N = 4$  documents

$N$  = total number of documents in the corpus

$\text{df}_t$  = number of documents containing term  $t$

| Term     | DF     | IDF computation                                 | IDF value | Interpretation                                  |
|----------|--------|-------------------------------------------------|-----------|-------------------------------------------------|
| the      | df = 4 | $\log_{10} (4/4) = \log_{10} (1) = 0$           | 0         | Appears in every document → least useful        |
| ball     | df = 2 | $\log_{10} (4/2) = \log_{10} (2) \approx 0.301$ | 0.301     | Appears in 2 of 4 documents → moderately useful |
| boundary | df = 1 | $\log_{10} (4/1) = \log_{10} (4) \approx 0.602$ | 0.602     | Appears in 1 of 4 documents → most useful       |

Direction to remember:

df ↑ ⇒ idf ↓

df ↓ ⇒ idf ↑

IDF converts corpus-wide commonness into a **down-weighting factor**.

Common terms get **low IDF**; document-specific terms get **high IDF**.

# TF-IDF: Combine the two ingredients

Final document weight = strength **HERE** × rarity **ELSEWHERE**

## Term Frequency (TF)

How strong is the term  
in this document?

×

## Inverse Document Frequency (IDF)

How rare is the term  
across the corpus?

=

## TF-IDF weight

$$\text{tfidf}(t,d) = \text{tf}(t,d) \times \text{idf}(t)$$

local frequency × corpus rarity

Example:  $D_1$  = “the batsman hit the ball to the boundary”

| Term     | Count<br>$c(t,d)$ | TF    | DF | IDF   | TF-IDF | Interpretation                   |
|----------|-------------------|-------|----|-------|--------|----------------------------------|
| the      | 3                 | 1.477 | 4  | 0     | 0      | Frequent here,<br>but everywhere |
| ball     | 1                 | 1     | 2  | 0.301 | 0.301  | Moderately useful                |
| boundary | 1                 | 1     | 1  | 0.602 | 0.602  | Most discriminative              |

TF-IDF is high for terms **frequent in this document** but **rare across the corpus**.

# Before and After: Raw Counts vs TF-IDF

TF-IDF uses the same vocabulary but has different weighting.

$D_1$ : "the batsman hit the ball to the boundary"

## Before: Bag-of-Words Counts

|          |   |
|----------|---|
| the      | 3 |
| ball     | 1 |
| boundary | 1 |

→  
Apply  
corpus-  
aware  
weighting

## After: TF-IDF Weights

|          |       |
|----------|-------|
| the      | 0     |
| ball     | 0.301 |
| boundary | 0.602 |

**Counts** : "the" seems important because it appears 3 times.

**TF-IDF** : "the" is common everywhere, so its weight should be near 0.

Rare but topic-revealing words survive.

TF-IDF changes the **values** of the coordinates, **not** the coordinate system itself.



# Where BoW / TF-IDF Still Fails

Weighting helps, but some important distinctions are still invisible.

## Word Order

"dog bites man"

"man bites dog"

Same words, very different meaning.

## Synonymy

"car"

"automobile"

Related meanings, different coordinates.

## Polysemy

"river bank"

"bank loan"

Same word form, different meanings.

## Composition / Context

"not good but funny"

"good but not funny"

Same unigram counts, different interpretation.

These cannot be corrected within this paradigm as they are representation limitations.

A representation can **only expose distinctions** that it chooses to encode.

# TF-IDF advantages and lacunae

Better weighting does not mean full understanding.

## What TF-IDF Improves

**Down-weights** words that occur in many documents.

**Up-weights** words that are frequent in the current document but rare elsewhere.

Gives a more **useful representation** for search and comparison.

Keeps the representation **fixed-length** and easy to compute.

the → low weight

## What TF-IDF Still Does Not Do

It still uses a **vocabulary-coordinate** representation.

It still ignores **word order**.

It still ignores **syntax and token positions**.

It still does not truly represent **meaning or context**.

dog bites man

≠

man bites dog

TF-IDF improves **feature weighting**; it does not yet solve **meaning**.

TF-IDF is a better **Bag-of-Words** representation, not a **semantic** representation.

# What This Representation Keeps — And What It Loses

Every representation makes modelling choices.

## Preserves

- Which vocabulary items occur
- How often they occur in a document
- Relative importance under TF-IDF weighting
- A fixed-dimensional vector for comparison and retrieval

## Collapses

- Different uses of the same word into one coordinate
- Example: bank in different senses
- Repeated structure into unordered counts

## Discards

- Word order
- Syntax
- Token positions
- Rich contextual meaning

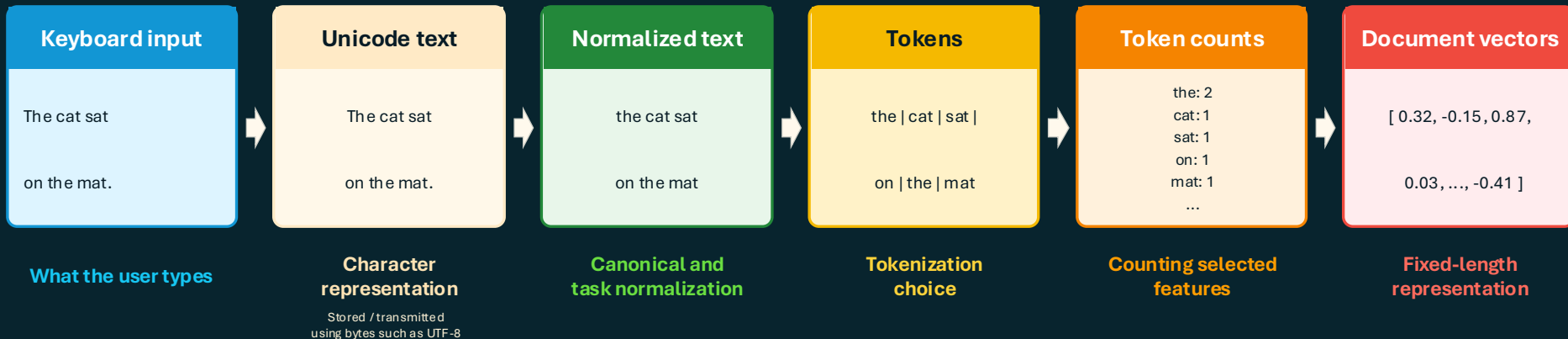
A representation decides **what to keep distinct**, **what to collapse**, and **what to discard**.

Its strengths and failure modes follow from those decisions.

BoW and TF-IDF are useful classical representations — but **they are not the end of the story**.

# Module 1 Synthesis: From Input Text to Document Representation

Representation is built step by step — and each step is a modelling choice.



## Key modelling choices in Module 1

- 1 **Encoding decides how written characters are represented.**
- 2 **Normalization decides which distinctions are made consistent or removed.**
- 3 **Tokenization decides what the basic computational units are.**
- 4 **Weighting decides how much importance each feature receives.**

What later models can learn depends on **what this representation makes visible.**

# Module 1: What You Should Now Be Able To Do

These are the core outcomes from Representing Language Computationally.

## Text and Characters

Distinguish words, morphemes, characters, and tokens.

Explain Unicode, code points, and UTF-8 at a conceptual level.

Explain why visible text and code-point sequences are not always the same.

Explain why normalization is needed.

## Tokenization

Explain the difference between Unicode normalization and task normalization.

Explain why whole-word tokenization is insufficient.

Describe the core idea of BPE.

Encode a simple unseen word using a learned BPE tokenizer.

## Document Representation

Construct a Bag-of-Words representation for a small document collection.

Compute TF, DF, IDF, and TF-IDF for simple examples.

Explain what BoW and TF-IDF preserve, collapse, and discard.

Interpret why weighting can improve whole-document comparison.

Module 1 has been about one Big Question:

**How should language be represented for computation?**

You should now be able to reason **from raw text** all the way **to weighted document vectors**.

# Module 1 Assessed Reading and Test Boundary

For the upcoming module test, you are responsible for the following material.

## Required Jurafsky–Martin Reading

### Chapter 2

- §2.1 Words
- §2.2 Morphemes
- §2.3 Unicode
- §2.4 Subword Tokenization: Byte-Pair Encoding
- §2.5 Corpora

### Chapter 11

- §11.1.1 Representing Documents as Vectors
- §11.1.2 Term Weighting

Stop at TF-IDF

Do not continue to §11.1.3 for this test

## Also Examinable From Lecture

- Unicode normalization, especially canonical equivalence and NFC.
- The distinction between Unicode normalization and task-specific normalization.
- Training a tokenizer versus using a fixed tokenizer for encoding.
- Character-level versus byte-level BPE at a conceptual level.
- Token fertility as an idea and why it matters.
- The course convention for TF and TF-IDF.
- What BoW and TF-IDF preserve, collapse, and discard.

The test covers **lectures**, **required reading**, and **the ideas in the coding assignments** given.

## Module 2

# Representing Language Computationally

Predicting Language and Linguistic Labels (Ch 3, J&M)

# From Representation to Prediction

Module 1 was about Representing language.    Module 2 is about Language Prediction.

## Document Representation

Document vector



**Predict a document label**

---

review → positive / negative

email → spam / not spam

---

*One whole document is treated  
as one object.*

Both are  
prediction  
problems.

## Token Sequence

Token history



**Predict the next token**

---

the batsman hit the \_\_\_\_

---

*The sequence itself becomes  
the predictive context.*

We begin Module 2 with the sequence question: **Given previous tokens, what comes next?**